Today 11:56 AM

02_air_hockey.zip
Zip Archive

Lab_4_VibeCoding_Stude...
PDF

Let us follow the guidelines and finish the assigned works. Can you tell me what are the errors in this project and in what order we will resolve them. We will commit changes one by one to my repo like they tell us to do.

Yes. Since there was a hiccup, the safest approach is to give you **one complete** `game/game_engine.py` containing the final versions of Tasks 2, 3, and 4, including the corrected goal handling.

The supplied README requires: scoring only when the puck fully passes through the goal gap, a 30-second match, and immediate puck continuation after a goal.

Lab_4_VibeCoding_Student_handout

`air-hockey/game/game_engine.py`

Replace the entire file with this:

```
"""
GameEngine: owns the puck, both paddles, and the computer AI, and runs
one frame's worth of game logic.

Implemented tasks:
- Task 1: puck-paddle collision physics
- Task 2: match scoring
- Task 3: 30-second match timer
- Task 4: post-goal puck reset and relaunch
"""

import math
import random

import pygame
```

```
from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import (
    WIDTH,
    HEIGHT,
    MARGIN,
    GOAL_TOP,
    GOAL_BOTTOM,
)

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5

# Task 3:
# The match lasts exactly 30 seconds.
MATCH_DURATION = 30


class GameEngine:
    def __init__(self):
        # Create the puck at the centre of the table.
        self.puck = Puck(
            WIDTH / 2,
            HEIGHT / 2,
            PUCK_RADIUS
        )

        # Give the puck an initial velocity.
        self._launch_puck()

        self.player = Paddle(
            x=WIDTH * 0.15,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS,
            max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )

        self.computer = Paddle(
            x=WIDTH * 0.85,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS,
            max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
```

```
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )

        self.ai = ComputerAI()

        # Task 2:
        # Maintain separate scores for the player and computer.
        self.player_score = 0
        self.computer_score = 0

        # Task 3:
        # Store the exact time at which the match begins.
        pygame_start_time = pygame.time.get_ticks()
        self.match_start_time = pygame_start_time

        # Task 3:
        # This becomes True once the 30-second match has ended.
        self.match_over = False

    def _launch_puck(self):
        """
        Give the puck a fresh velocity for the beginning of a point.

        A random horizontal direction and vertical angle prevent every
        point from starting with exactly the same trajectory.
        """

        angle_choices = [0.3, 0.6, -0.3, -0.6]

        direction = random.choice([-1, 1])
        vy_factor = random.choice(angle_choices)

        self.puck.vx = INITIAL_PUCK_SPEED * direction
        self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

    def _get_elapsed_time(self):
        """
        Return the number of seconds that have elapsed since the
        match started.
        """

        return (
            pygame.time.get_ticks() - self.match_start_time
        ) / 1000.0

    def _get_remaining_time(self):
        """
        Return the number of whole seconds remaining in the match.

        ceil() keeps the display at 30 at the start of the match and
        decreases it once each whole second has elapsed.
        """
```

```
        elapsed_seconds = self._get_elapsed_time()

        return max(
            0,
            math.ceil(MATCH_DURATION - elapsed_seconds)
        )

    def handle_input(self, keys_pressed):
        """
        Handle keyboard input for the player paddle.

        Input is ignored once the match has ended.
        """

        # Task 3:
        # Do not allow the player to move after time expires.
        if self.match_over:
            return

        # Check the timer here as well as in update().
        # This prevents the player from moving during the frame in
        # which the timer first reaches zero.
        if self._get_elapsed_time() >= MATCH_DURATION:
            self.match_over = True
            return

        dx = 0
        dy = 0

        if keys_pressed[pygame.K_UP]:
            dy -= PLAYER_SPEED

        if keys_pressed[pygame.K_DOWN]:
            dy += PLAYER_SPEED

        if keys_pressed[pygame.K_LEFT]:
            dx -= PLAYER_SPEED

        if keys_pressed[pygame.K_RIGHT]:
            dx += PLAYER_SPEED

        self.player.move_by(dx, dy)

    def update(self):
        """
        Update one frame of the game.

        Timer expiration is checked before any gameplay logic so that
        the match stops when the 30-second limit is reached.
        """
```

```
        # Task 3:
        # End the match once 30 seconds have elapsed.
        if self._get_elapsed_time() >= MATCH_DURATION:
            self.match_over = True
            return

        # Safety check in case update() is called after the match ends.
        if self.match_over:
            return

        # Update computer AI.
        self.ai.update(self.computer, self.puck)

        # Move the puck.
        self.puck.move()

        # Handle top and bottom wall collisions.
        self.puck.bounce_off_walls(HEIGHT, MARGIN)

        # Task 1:
        # Use the collision system to handle paddle impacts.
        handle_paddle_collision(
            self.puck,
            self.player
        )

        handle_paddle_collision(
            self.puck,
            self.computer
        )

        # Task 2:
        # Check for goals and side-wall collisions.
        self._handle_goals()

    def _handle_goals(self):
        """
        Handle goals and collisions with the left/right end walls.

        A point is scored only when the entire puck fits through the
        goal opening and completely crosses the end of the table.

        If the puck reaches a wall outside the goal opening, it
        bounces normally instead.
        """

        # The complete puck must fit vertically inside the goal gap.
        #
        # Simply checking the puck's centre is not enough. This prevents
        # the puck from being counted as a goal while still touching
        # the goalpost.
        puck_fits_in_goal = (
```

```
        GOAL_TOP + self.puck.radius
        <= self.puck.y
        <= GOAL_BOTTOM - self.puck.radius
    )

    # ----------------------------------------------------------
    # LEFT SIDE
    # ----------------------------------------------------------
    if self.puck.vx < 0:

        if puck_fits_in_goal:
            # The puck is inside the goal opening.
            #
            # Do NOT bounce it at MARGIN. The goal is an opening
            # in the end wall, so the puck is allowed to travel
            # through it.
            #
            # Register the goal only after the entire puck has
            # crossed the left edge of the screen.
            if self.puck.x + self.puck.radius <= 0:

                # The puck entered the player's goal,
                # therefore the computer receives the point.
                self.computer_score += 1

                # Reset the puck for the next point.
                self._reset_puck()
                return

        else:
            # Outside the goal opening, the left end wall behaves
            # like a normal wall.
            if self.puck.x - self.puck.radius < MARGIN:

                self.puck.x = MARGIN + self.puck.radius
                self.puck.vx = -self.puck.vx
                return

    # ----------------------------------------------------------
    # RIGHT SIDE
    # ----------------------------------------------------------
    elif self.puck.vx > 0:

        if puck_fits_in_goal:
            # The puck is inside the goal opening, so allow it
            # to travel through the opening.
            #
            # Register the goal only after the entire puck has
            # crossed the right edge of the screen.
            if self.puck.x - self.puck.radius >= WIDTH:

                # The puck entered the computer's goal,
```

```
                # therefore the player receives the point.
                self.player_score += 1

                # Reset the puck for the next point.
                self._reset_puck()
                return

            else:
                # Outside the goal opening, the right end wall
                # behaves like a normal wall.
                if self.puck.x + self.puck.radius > WIDTH - MARGIN:

                    self.puck.x = (
                        WIDTH - MARGIN - self.puck.radius
                    )
                    self.puck.vx = -self.puck.vx
                    return

    def _get_match_result(self):
        """
        Determine the final match result from the current scores.
        """

        if self.player_score > self.computer_score:
            return "Player wins!"

        if self.computer_score > self.player_score:
            return "Computer wins!"

        return "Draw!"

    def _reset_puck(self):
        """
        Reset the puck to the centre and immediately launch it for
        the next point.

        Any previous-position state used by the collision system is
        also reset so that the new point starts from a clean state.
        """

        # Put the puck exactly at the centre.
        self.puck.x = WIDTH / 2
        self.puck.y = HEIGHT / 2

        # Task 1 compatibility:
        # If the puck stores its previous position for continuous
        # collision detection, reset it too. Otherwise the collision
        # system could incorrectly think the puck travelled from the
        # previous goal to the centre in one frame.
        if hasattr(self.puck, "previous_x"):
            self.puck.previous_x = self.puck.x
```

```
            if hasattr(self.puck, "previous_y"):
                self.puck.previous_y = self.puck.y

            # Task 4:
            # Immediately give the puck a fresh speed and direction.
            # This prevents it from becoming stationary after a goal.
            self._launch_puck()

        def draw(self, surface, font):
            """
            Draw the complete game state.
            """

            from game import renderer

            renderer.draw_table(surface)

            renderer.draw_paddle(
                surface,
                self.player,
                renderer.COLOR_PLAYER
            )

            renderer.draw_paddle(
                surface,
                self.computer,
                renderer.COLOR_COMPUTER
            )

            renderer.draw_puck(
                surface,
                self.puck
            )

            # ------------------------------------------------------------
            # Task 2: SCORE DISPLAY
            # ------------------------------------------------------------

            player_text = f"Player: {self.player_score}"
            computer_text = f"Computer: {self.computer_score}"

            renderer.draw_text(
                surface,
                font,
                player_text,
                (MARGIN + 10, MARGIN + 8)
            )

            computer_width = font.size(computer_text)[0]

            renderer.draw_text(
                surface,
```

```
            font,
            computer_text,
            (
                WIDTH - MARGIN - computer_width - 10,
                MARGIN + 8
            )
        )

        # ------------------------------------------------------------
        # Task 3: TIMER DISPLAY
        # ------------------------------------------------------------

        time_text = f"Time: {self._get_remaining_time()}"
        time_width = font.size(time_text)[0]

        renderer.draw_text(
            surface,
            font,
            time_text,
            (
                (WIDTH - time_width) // 2,
                MARGIN + 8
            )
        )

        # ------------------------------------------------------------
        # Task 3: FINAL RESULT
        # ------------------------------------------------------------

        if self.match_over:
            renderer.draw_banner(
                surface,
                font,
                self._get_match_result()
            )
```

Why this version fixes the goal problem

The important correction is in `_handle_goals()`.

The opening occupies the middle section of each end wall. When the puck is inside that opening, we **do not bounce at** `MARGIN`. Instead, we let it continue until the entire puck crosses the actual screen edge:

Python Run

for the left goal, and:

Python Run

for the right goal.

Outside the opening, the wall still behaves normally and the puck bounces.

That matches the assignment's expected behavior: a goal is counted only after the puck fully passes through the goal gap, while hitting the wall elsewhere produces a bounce.

Lab_4_VibeCoding_Student_handout

After replacing the file

Run:

Bash

Test a shot through the **middle of the goal opening** first. The puck should pass through rather than bounce, the appropriate score should increment, and the puck should immediately reappear at centre with a new trajectory.

One correction to my earlier guidance: because your actual `game_engine.py` had already diverged from the starter, this full-file version is based on the **supplied project's actual structure** rather than pretending I can see your latest local edits. You should preserve any unrelated changes you intentionally made elsewhere.

Sources

c162937c-8b2f-47a2-8d8...
Python

Is the attached main.py correct? because the puck goes into the goal and the goal is registered, but the puck is not being placed in the center afterwards. also the R does not work. Let us fix this and commit.

Worked for 1m 1s

No. The attached `main.py` is still the **starter version**. It handles `QUIT`, reads the arrow keys, updates the engine, and draws the frame, but there is no `KEYDOWN` handling for `R`.

c162937c-8b2f-47a2-8d8b-4e7845e...

The center-reset issue is separate: it belongs in `game_engine.py`. Because Task 4 requires the puck to reset to the center **and immediately continue**, `_reset_puck()` should place it at center, reset any stored previous-position state, and then relaunch it.

Lab_4_VibeCoding_Student_handout

1. Replace `main.py` with this

```
"""
Air Hockey (Lab Starter)

Run with:  python3 main.py

Controls:
- Arrow keys move your paddle (left side, blue).
- Press R at any time to restart the match.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()

    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Air Hockey")

    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 24)

    engine = GameEngine()
    running = True

    while running:
        for event in pygame.event.get():

            if event.type == pygame.QUIT:
                running = False

            # Restart the complete match when R is pressed.
            # Creating a new GameEngine resets:
```

```
            # - player score
            # - computer score
            # - 30-second timer
            # - puck position
            # - paddle positions
            # - match-over state
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    engine = GameEngine()

        keys = pygame.key.get_pressed()

        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

The important addition is:

```
elif event.type == pygame.KEYDOWN:
    if event.key == pygame.K_r:
```

Python Run

Recreating the engine is preferable here because it automatically starts a completely fresh match, including a fresh 30-second timer.

2. Fix `_reset_puck()` in `game_engine.py`

Use this version:

```
def _reset_puck(self):
    """
    Reset the puck to the centre of the table and immediately launch it
    for the next point.

    Any previous-position state used by the collision system is also
```

```
        reset so the new point starts cleanly.
        """

        # Place the puck exactly at the centre of the table.
        self.puck.x = WIDTH / 2
        self.puck.y = HEIGHT / 2

        # Task 1 compatibility:
        # Reset the previous position as well, if it exists.
        # Otherwise the collision system could interpret the new point
        # as the puck travelling directly from the goal to the centre.
        if hasattr(self.puck, "previous_x"):
            self.puck.previous_x = self.puck.x

        if hasattr(self.puck, "previous_y"):
            self.puck.previous_y = self.puck.y

        # Task 4:
        # Immediately give the puck a fresh velocity and direction.
        self._launch_puck()
```

Why your "not centered" observation can happen

Technically, with:

```
self.puck.x = WIDTH / 2
self.puck.y = HEIGHT / 2
```

Python Run

the puck **does get placed at the centre**. It then immediately starts moving, so at 60 FPS you may barely notice the center position.

That is actually consistent with the assignment's expected behavior: after every goal, the puck should immediately continue from the center rather than sitting motionless.

Lab_4_VibeCoding_Student_handout

The `previous_x` / `previous_y` reset is the important extra piece because of the Task 1 continuous-collision implementation.

Final test

Run:

Bash

Test these three things:

Goal:
Puck enters the goal → score increments → puck restarts from centre with a new trajectory.

Restart:
Press `R` → scores return to `0–0`, timer returns to `30`, puck starts from centre, and the match is active again.

Restart after match end:
Let timer reach `0`, then press `R` → new match begins.

Now commit

Since we're done with the four assigned tasks plus this small README-controls correction, check the complete diff first:

```
git status
```

Bash

Then stage the relevant project changes:

Bash

Commit:

Bash

Then push to your fork:

Bash

I would **not** make another Task 5 out of `R`; the README already specifies `R` as a control, so this is simply completing the advertised behavior.

Final tally

4 / 4 assigned tasks completed.

The only remaining assignment work is the **post-fix 10-second video** and exporting the **complete Chat/LLM history** for submission; the handout lists those along with the updated code. Lab_4_VibeCoding_Student_handout

Sources

Great! Everything is working and R works too. All good. Now I have the pre-game recording in my repo folder and post game too. I just need to commit an appropriate message and push all the stuff to my repo.

Perfect. Since everything is tested and the pre/post recordings are already inside the repo folder, you can now make the final submission commit.

I recommend this commit message:

```
feat: complete air hockey lab assignment
```

From your cloned repository:

Bash

Make sure you see your modified source files and both videos.

Then:

Bash

Check what is actually staged:

Bash

Then commit:

Bash

Finally push to your fork:

Bash

One important check

Before pushing, make sure the videos are actually tracked:

Bash

```
git status
```

If they appear under “Changes to be committed,” you're good.

You can also verify the final commit with:

Bash

```
git log --oneline -1
```

You should see something like:

```
abc1234 feat: complete air hockey lab assignment
```

And because the lab explicitly asks for the **before video, after video, updated code, and exported chat history**, make sure the chat-history file is also in your `Lab-4` submission folder before `git add -A`. Lab_4_VibeCoding_Student_handout

After `git push origin main` completes, your fork will contain the complete assignment.

Sources

ChatGPT can make mistakes. Check important info.